Music Playlist Manager – Sprint 1 Backlog

Software Architecture Design 2026

Corso di Laurea Magistrale in Ingegneria Informatica
Anno Accademico 2025/2026



Docenti: Ritrovato Pierluigi, Apicella Andrea

GitHub: `https://github.com/fra343/Gruppo2-SAD`

Trello: `https://trello.com/b/Mpb9bjYE/sad`

Gruppo 02:

Apicella Francesco	IE22700106
Campagna Lorenzo	IE22700105
Cutolo Raffaele	IE22700113
Gioia Errico	IE22700112

Contents

1 Introduzione **2**
1.1 Obiettivo dello Sprint . . . 2
1.2 Stima della velocity . . . 2
1.3 User Stories selezionate . . . 2

2 Task Breakdown e assegnazioni **3**
2.1 Legenda e scala di stima . . . 3
2.2 US 0 – Setup condiviso (9 ore-uomo totali) . . . 3
2.3 Lorenzo Campagna (LC) – Carico totale: 9h . . . 3
2.4 Francesco Apicella (FA) – Carico totale: 9h . . . 4
2.5 Raffaele Cutolo (RC) – Carico totale: 9h . . . 4
2.6 Errico Gioia (EG) – Carico totale: 8h . . . 5

3 Assegnazione delle code review **5**

4 Dettaglio delle User Stories **6**
4.1 US 1: Creare una traccia *(Assegnato a: Lorenzo Campagna)* . . . 6
4.2 US 2: Modificare una traccia *(Assegnato a: Francesco Apicella)* . . . 6
4.3 US 3: Eliminare una traccia *(Assegnato a: Raffaele Cutolo)* . . . 6
4.4 US 4: Visualizzare la lista delle tracce *(Assegnato a: Raffaele Cutolo)* . . . 7
4.5 US 5: Creare ed eliminare una playlist *(Assegnato a: Francesco Apicella)* . . . 7
4.6 US 6: Aggiungere e rimuovere tracce da una playlist *(Assegnato a: Errico Gioia)* . . . 7

1 Introduzione

Questa sezione documenta la pianificazione del primo sprint. Lo Sprint 1 inaugura il ciclo di sviluppo del prodotto e ha l'obiettivo di costruire le fondamenta architetturali su cui gli sprint successivi aggiungeranno funzionalità incrementali.

1.1 Obiettivo dello Sprint

Realizzare il nucleo principale dell'applicazione, ovvero la gestione delle tracce musicali, la creazione delle playlist e l'associazione tra tracce e playlist.
Al termine dello Sprint l'utente deve poter aggiungere, modificare visivamente, eliminare e visualizzare tracce nella libreria, creare ed eliminare playlist e popolarle con le tracce della libreria.

1.2 Stima della velocity

Trattandosi del primo Sprint il team non dispone di dati storici sulla propria velocity. Si è quindi adottata una stima prudente ma non minimale, basata sulla capacity oraria disponibile. Su questa base sono stati selezionati **11 Story Points**, per circa **35 ore-uomo** pianificate, lasciando circa 1h di margine sulla capacity di 36h. La velocity reale sarà misurata in sede di Sprint Review e utilizzata come riferimento per gli sprint successivi.
La capacity dello Sprint è stata determinata considerando 9 ore-uomo settimanali per ciascuno dei 4 membri del team, per un totale di **36 ore-uomo**.

1.3 User Stories selezionate

Sono state scelte sei user stories di Alta priorità, posizionate in cima al Product Backlog e tra loro fortemente coese sul piano funzionale (gestione di base di tracce e playlist). Le US 1, 2, 3 e 4 coprono in modo completo le operazioni di gestione delle tracce (creazione, modifica, eliminazione e visualizzazione); le US 5 e 6 introducono l'entità Playlist e le operazioni essenziali per popolarla. Le US relative a riproduzione e UI real-time sono volutamente rinviate allo Sprint 2 perché poggiano sulle entità di dominio che verranno introdotte in questo Sprint.

#	**User Story**	**SP**
US-01	Come utente voglio aggiungere una traccia specificando titolo, autore, durata, genere e anno, così da popolare la mia libreria musicale.	2
US-02	Come utente voglio modificare i dati di una traccia esistente, così da correggere errori o aggiornare informazioni.	2
US-03	Come utente voglio eliminare una traccia, così da rimuovere brani che non mi interessano più.	1
US-04	Come utente voglio visualizzare la lista di tutte le tracce, così da avere una vista completa della mia libreria.	2
US-05	Come utente voglio creare ed eliminare una playlist assegnandole un nome, così da poter organizzare le tracce in raccolte tematiche.	1
US-06	Come utente voglio aggiungere e rimuovere tracce da una playlist, così da personalizzarne il contenuto.	3
	Totale Story Points	**11**

Table 1: User Stories selezionate per lo Sprint 1.

2 Task Breakdown e assegnazioni

L'organizzazione del lavoro segue un approccio a *ownership per user story*: ciascuna US viene presa in carico da un singolo membro che la porta dall'inizio alla fine, attraversando tutti i layer architetturali (Model, Service, Controller, View, Test). Questa scelta è motivata da tre fattori: maggiore tracciabilità individuale, maggiore resilienza in caso di indisponibilità di un membro, e maggiore valore formativo per ciascuno dei partecipanti, che si troverà a toccare l'intero stack tecnologico anziché un solo layer. Questo approccio presuppone tuttavia che alcune classi del Model (`Track`, `Playlist`, `Library`) e alcune componenti di infrastruttura (interfaccia `Command`, `CommandHistory`, scheletro dei service e dei controller) siano definite collettivamente prima che ciascun membro inizi il lavoro autonomo sulla propria US.

2.1 Legenda e scala di stima

Legenda: FA = Francesco Apicella – LC = Lorenzo Campagna – RC = Raffaele Cutolo – EG = Errico Gioia.
Le ore di sviluppo sono state stimate seguendo una scala di riferimento interna: **1 SP = 2.5h**, **2 SP = 4h**, **3 SP = 6h**. La progressione non è perfettamente lineare per riflettere la maggiore incertezza delle US più ampie.

2.2 US 0 – Setup condiviso (9 ore-uomo totali)

Lavoro preparatorio distribuito tra i membri prima dello split sulle singole US:

- Configurazione `pom.xml` con dipendenze JavaFX, JUnit 5, Jackson (LC, 1h);
- Scheletro dei package `model`, `service`, `controller`, `view`, `playback`, `command`, `persistence` (FA, 0.5h);
- Setup base dei service (`LibraryService`, `PlaylistService`) con firme dei metodi pubblici (RC, 0.5h);
- Setup di `Main.fxml` e `MainController` con shell vuota dell'interfaccia (LC, 1h);
- Definizione interfaccia `Command` e classe `CommandHistory` (EG, 1h);
- Sessione di design collettiva: classi `Track`, `Playlist`, `Library`, scelta di `ObservableList<Track>`, contratti degli helper (tutti, 1h ciascuno = 4h ore-uomo);
- Implementazione `ValidationUtils` (utility statica per validare anno, durata, stringhe non vuote) (LC, 1h).

2.3 Lorenzo Campagna (LC) – Carico totale: 9h

Setup condiviso (4h).

- Configurazione `pom.xml` con dipendenze (1h);
- Setup `Main.fxml` e `MainController` shell (1h);
- Implementazione `ValidationUtils` (1h);
- Partecipazione alla sessione di design collettiva (1h).

US 1 – Creare una traccia (2 SP, 4h).

- Completamento classe `Track` e `LibraryService.addTrack` con validazione tramite `ValidationUtils` (1.5h);
- Test JUnit su `Track` e `LibraryService.addTrack` (0.5h);
- FXML del form di inserimento e `TrackListController.onAddTrack` con gestione errori (2h).

Rifinitura e code review (1h).

- Javadoc su `Track`, `LibraryService.addTrack`, `ValidationUtils` + test sul codice US 1 (0.5h);
- Code review di US 6 (EG) (0.5h).

2.4 Francesco Apicella (FA) – Carico totale: 9h

Setup condiviso (1.5h).

- Creazione scheletro dei package (0.5h);
- Partecipazione alla sessione di design collettiva (1h).

US 2 – Modificare una traccia (2 SP, 4h).

- `LibraryService.updateTrack` e setter sui campi di `Track` (1h);
- Test JUnit su `LibraryService.updateTrack` (modifica singola, multipla, valori non validi) (1h);
- Adattamento del form FXML per modalità Edit e `TrackListController.onEditTrack` con precompilazione (2h).

US 5 – Creare ed eliminare una playlist (1 SP, 2.5h).

- Completamento classe `Playlist` (`addTrack`, `removeTrack`, gestione interna dei dati) (1h);
- `PlaylistService.createPlaylist` con controllo nomi duplicati, `deletePlaylist` e UI minimale (dialog + lista laterale) (1h);
- Test JUnit per `createPlaylist` e `deletePlaylist` (0.5h).

Rifinitura e code review (1h).

- Javadoc su `Playlist`, `LibraryService.updateTrack`, `PlaylistService` + test sul codice US 2 e US 5 (0.5h);
- Code review di US 1 (LC) (0.5h).

2.5 Raffaele Cutolo (RC) – Carico totale: 9h

Setup condiviso (1.5h).

- Setup base dei service con firme dei metodi pubblici (0.5h);
- Partecipazione alla sessione di design collettiva (1h).

US 3 – Eliminare una traccia (1 SP, 2.5h).

- `LibraryService.removeTrack` con propagazione alle playlist contenenti (1h);
- Test JUnit su `LibraryService.removeTrack` (inclusa la rimozione propagata) (0.5h);
- `TrackListController.onDeleteTrack` con dialog di conferma (1h).

US 4 – Visualizzare la lista delle tracce (2 SP, 4h).

- Completamento `Library` con `ObservableList<Track>` e `LibraryService.getAllTracks` (1h);
- Test JUnit su `Library` e `LibraryService.getAllTracks` (lista vuota, una traccia, molte tracce) (1h);
- FXML della vista lista tracce (`ListView` con cell factory minimale) e binding `TrackListController` $\leftrightarrow$ service (2h).

Rifinitura e code review (1h).

- Javadoc su `LibraryService.removeTrack`, `LibraryService.getAllTracks`, controller delle viste lista + test sul codice US 3 e US 4 (0.5h);
- Code review di US 2 e US 5 (FA) (0.5h).

2.6 Errico Gioia (EG) – Carico totale: 8h

Setup condiviso (2h).

- Definizione interfaccia `Command` e classe `CommandHistory` (1h);
- Partecipazione alla sessione di design collettiva (1h).

US 6 – Aggiungere e rimuovere tracce da una playlist (3 SP, 5h).

- `PlaylistService.addTrackToPlaylist` e `removeTrackFromPlaylist` (1.5h);
- `AddTrackToPlaylistCommand` e `RemoveTrackFromPlaylistCommand` con `execute` e implementazione del metodo `push` della classe `CommandHistory` (già predisposta nella configurazione iniziale) per registrare i comandi eseguiti. L'undo lasciato come placeholder non implementato (TODO rinviato a US-12) (1.5h);
- Test JUnit per i due metodi di service (`addTrackToPlaylist` e `removeTrackFromPlaylist`) e per il metodo `execute` di `AddTrackToPlaylistCommand` e `RemoveTrackFromPlaylistCommand`. Nessun test di `undo()` (rinviato a US-12) (1h);
- UI per aggiunta/rimozione tracce (selezione dalla libreria, bottoni Add/Remove) (1h).

Rifinitura e code review (1h).

- Javadoc su `PlaylistService` (metodi add/remove), `Command`, `CommandHistory` + test sul codice US 6 (0.5h);
- Code review di US 3 e US 4 (RC) (0.5h).

La US6 risulta essere da 5h nonostante i 3 SP: con setup (2h) e rifinitura (1h), EG chiude quindi lo Sprint 1 a 8h invece di 9h come i colleghi. L'ora mancante è il lavoro di undo che si trasferisce alla US-12 poiché la seguente funzionalità ha priorità media e sarà scelta per una sprint futura; eseguirla ora avrebbe violato lo scope della sprint.

3 Assegnazione delle code review

Nel modello *vertical slice* il rischio principale è che ogni US resti confinata a un solo membro, con scarsa conoscenza incrociata del codice prodotto. Per contrastare questo rischio si assegnano esplicitamente le code review in modo che ogni membro riveda il lavoro di un altro:

Owner della US	Reviewer
LC (US 1)	FA
FA (US 2, US 5)	RC
RC (US 3, US 4)	EG
EG (US 6)	LC

Table 2: Assegnazione delle code review fra i membri del team.

4 Dettaglio delle User Stories

4.1 US 1: Creare una traccia *(Assegnato a: Lorenzo Campagna)*

- Implementare il completamento della classe `Track` e il metodo `LibraryService.addTrack`, assicurando la validazione dei dati tramite `ValidationUtils`.
- Creare il file FXML per il form di inserimento della traccia musicale.
- Implementare il metodo `TrackListController.onAddTrack` gestendo opportunamente gli errori di inserimento per l'interfaccia.
- Scrivere i test JUnit per validare il comportamento della classe `Track` e del metodo `LibraryService.addTrack`.
- Compilare la Javadoc per la classe `Track`, il metodo `LibraryService.addTrack` e la classe di utilità `ValidationUtils`.

4.2 US 2: Modificare una traccia *(Assegnato a: Francesco Apicella)*

- Implementare il metodo `LibraryService.updateTrack` e i relativi metodi setter sui campi della classe `Track`.
- Adattare il form FXML esistente per supportare graficamente la modalità di modifica (Edit).
- Sviluppare il metodo `TrackListController.onEditTrack` includendo la logica di precompilazione dei campi della UI.
- Creare i test JUnit per `LibraryService.updateTrack`, assicurandosi di coprire i casi di modifica singola, modifica multipla e inserimento di valori non validi.
- Verificare, tramite test JUnit, che dopo `updateTrack` i valori aggiornati si riflettano automaticamente anche in una playlist che contiene la traccia modificata.
- Redigere la Javadoc documentando il funzionamento del metodo `LibraryService.updateTrack`.

4.3 US 3: Eliminare una traccia *(Assegnato a: Raffaele Cutolo)*

- Implementare il metodo `LibraryService.removeTrack`, programmando la logica affinché la rimozione si propaghi correttamente in tutte le playlist che contengono la traccia.
- Sviluppare il metodo `TrackListController.onDeleteTrack` implementando e richiamando un dialog UI di conferma per l'utente.
- Scrivere i test JUnit per `LibraryService.removeTrack`, includendo esplicitamente la verifica della propagazione della rimozione all'interno delle playlist.
- Aggiungere la documentazione Javadoc per il metodo `LibraryService.removeTrack`.

Nota di scope: lo scenario 2 di US-03 (eliminazione di una traccia in riproduzione, con interruzione e avanzamento automatico alla successiva) dipende dal `PlaybackService` e sarà coperto nello sprint che introduce la riproduzione; in questo sprint si implementa e verifica solo lo scenario 1 (traccia non in riproduzione).

4.4 US 4: Visualizzare la lista delle tracce *(Assegnato a: Raffaele Cutolo)*

- Completare la classe `Library` integrandovi una `ObservableList<Track>` per sfruttare le property di JavaFX.
- Implementare il metodo `LibraryService.getAllTracks` per recuperare i dati della libreria.
- Creare la vista FXML per la lista delle tracce utilizzando il componente `ListView` e definendo una cell factory minimale.
- Realizzare il binding dei dati tra il `TrackListController` e il service corrispondente per aggiornare la view.
- Gestire nella view lo stato di libreria vuota, mostrando un messaggio che informa l'utente e lo invita ad aggiungere una traccia (scenario 2 di US-04).
- Sviluppare i test JUnit per la classe `Library` e il metodo `LibraryService.getAllTracks`, testando specificamente i casi con lista vuota, con singola traccia e con tracce multiple.
- Redigere la Javadoc per `LibraryService.getAllTracks` e per i controller legati alle viste della lista.

4.5 US 5: Creare ed eliminare una playlist *(Assegnato a: Francesco Apicella)*

- Completare la logica interna della classe `Playlist`, implementando i metodi `addTrack`, `removeTrack` e la corretta gestione interna dei dati.
- Implementare i metodi `PlaylistService.createPlaylist` inserendo un controllo logico per impedire la creazione di nomi duplicati.
- Implementare il metodo `PlaylistService.deletePlaylist`.
- Creare una UI minimale in JavaFX che includa un dialog per la creazione e una lista laterale per la visualizzazione delle playlist.
- Scrivere i test JUnit per verificare l'affidabilità di `createPlaylist` e `deletePlaylist`.
- Aggiungere la Javadoc per l'intera classe `Playlist` e i relativi metodi all'interno del `PlaylistService`.

4.6 US 6: Aggiungere e rimuovere tracce da una playlist *(Assegnato a: Errico Gioia)*

- Implementare i metodi `addTrackToPlaylist` e `removeTrackFromPlaylist` all'interno della classe `PlaylistService`.
- Sviluppare le classi `AddTrackToPlaylistCommand` e `RemoveTrackFromPlaylistCommand` implementando rigorosamente solo il metodo `execute` (il metodo `undo` deve restare come TODO vuoto per la US-12).
- Implementare il metodo `push` all'interno della classe `CommandHistory` allo scopo di registrare storicamente la pila di comandi eseguiti.
- Creare l'interfaccia utente (UI) per consentire la selezione visiva delle tracce dalla libreria e collegare i bottoni "Add" e "Remove".
- Scrivere i test JUnit esclusivamente per i due metodi del service e per il metodo `execute` dei due Command (omettere i test per la logica di undo).
- Redigere la Javadoc per i nuovi metodi in `PlaylistService`, per l'interfaccia `Command` e per `CommandHistory`.